VisionInspect AI Milestone 3

VISIONINSPECT AI

AI/ML MODULE | MILESTONE 3

**Defect Subtype Classification,
Severity Scoring, and QA Decisions**

Category-specific classifier runtimes, confidence separation, weighted severity, conservative release policy, and practical decision evidence

Submission field	Details
Project	VisionInspect AI
Assigned module	Member 4 - AI/ML
Prepared by	Himanshu Sekhar Parhi
Mentor	Sai Jyoteesh
Programme	Infosys Springboard Programme
Document version	Milestone 3 Submission - Version 1.0
Submission date	8 August 2026

Document Control

This report records Milestone 3 of the VisionInspect AI Member 4 module. It documents how a detected anomaly is assigned a category-specific defect subtype, how evidence is converted into a weighted severity score, and how the runtime produces a conservative Pass, Review, or Fail decision.

Submission sequence: Milestone 1 - 25 July 2026; Milestone 2 - 1 August 2026; Milestone 3 - 8 August 2026; Milestone 4 - 15 August 2026. This report is the third practical stage in that weekly progression.

> **Milestone status:** Completed. Every supported category has a defect-only subtype classifier artifact and recorded metrics; all classifier labels have explicit severity mappings; confidence values remain separated; and focused classifier, severity, prediction, and preprocessing tests pass.

Control item	Recorded value
Milestone	3 - Defect Subtype Classification, Severity Scoring, and QA Decisions
Primary notebooks	04_defect_subtype_classification.ipynb; 06_severity_and_qa_decision.ipynb
Classifier coverage	15 categories; 73 category-label pairs; 48 unique defect subtype names
Runtime engines	2 fine-tuned ResNet18 ONNX; 1 portable forest; 12 scikit-learn bundles
Decision policy	Severity weights 30/25/25/20; Review threshold 40; Fail threshold 60
Focused tests	20 passed in 4.86 seconds
Repository branch	Himanshu-parhi-ai/ml

Milestone Snapshot

Value	Meaning
15	Supported categories with classifier artifacts
73	Category-specific subtype labels
48	Unique defect subtype names
0.8925	Mean recorded classifier accuracy
0.8931	Mean recorded classifier macro F1
53	Explicit defect risk mappings, including aliases and fallback labels
20	Focused automated tests passed

Table of Contents

List of Figures and Key Terms

Key terms

Term	Meaning in VisionInspect AI
Anomaly detection	Determines whether the product is Good or Defective.
Subtype classification	Names a known defect only after a defect has been detected.
Detection confidence	Confidence in the binary anomaly decision; used by severity.
Subtype confidence	Confidence in the selected defect label; used for interpretation.
Severity	Weighted quality-impact score from 0 to 100.
QA decision	Operational Pass, Review, or Fail result.
Unknown defect	Conservative label used when subtype evidence is unavailable or conflicting.

1. Executive Summary

Milestone 3 converts anomaly evidence into an interpretable manufacturing outcome. The binary anomaly detector remains responsible for Good versus Defective. Only defective images proceed to a category-specific subtype classifier. That classifier names one known failure pattern, returns class probabilities, and keeps its confidence separate from the anomaly detector's confidence. The resulting evidence is combined with defect area and critical-zone overlap to produce a severity score and operational quality decision.

Classifier records cover all 15 MVTec categories, 73 category-label pairs, and 48 unique subtype names. The unweighted mean recorded accuracy is 0.8925 and mean macro F1 is 0.8931. Two weak categories, capsule and wood, use fine-tuned ResNet18 classifiers exported to ONNX. Cable uses a 173 KB portable forest. The remaining categories use promoted scikit-learn pipelines with shared ResNet18 ONNX features and engineered image evidence.

The severity framework implements the project formula exactly: size 30%, location 25%, defect type 25%, and detection confidence 20%. Scores from 0-39 are Low, 40-59 Medium, 60-79 High, and 80-100 Critical. The runtime then applies a conservative release rule: severity at least 60 fails, severity at least 40 requires review, and any detected anomaly below 40 also requires review instead of being automatically passed.

> **Practical evidence:** Notebook 04 demonstrates correct subtype predictions for cable, capsule, bottle, and wood using three runtime engines. Notebook 06 demonstrates a real broken-large bottle result with severity 79.02 (High) and a Fail decision.

1.1 Key outcomes

- Defect subtype classification is performed only after a binary defect is detected.
- The category registry selects the correct classifier artifact and runtime engine.
- Classifier evaluation uses macro F1, class reports, and confusion matrices rather than accuracy alone.
- Detection confidence and subtype confidence remain distinct throughout inference.
- Every active classifier label has an explicit severity risk score.
- A conservative unknown_defect path prevents unsupported subtype claims.
- Operational Pass, Review, and Fail decisions are derived from configurable severity thresholds.

2. Milestone Scope and Acceptance Criteria

2.1 Objective

The Milestone 3 objective is to classify known defect subtypes after anomaly detection, preserve uncertainty honestly, score product impact consistently, and generate a quality-control action that can be consumed by the application workflow.

2.2 Included work

- Defect-only labelled dataset construction from MVTec test defect folders.
- Category-specific feature engineering and classifier model selection.
- Fine-tuned ResNet18 classifiers for capsule and wood, exported to ONNX.
- Portable forest export for cable and scikit-learn bundle support for other categories.
- Class probabilities, subtype confidence, and unknown-defect fallback behavior.

- Weighted severity components, levels, recommendations, and QA decision thresholds.
- Practical subtype and decision demonstrations using real MVTec images.

2.3 Excluded or deferred work

- Binary anomaly training and heatmap localization belong to Milestone 2.
- Backend persistence, batch processing, frontend review, reports, and deployment belong to Milestone 4.
- The classifier does not discover a new semantic subtype outside its trained label set.
- Severity is an engineering policy score, not a certified safety or regulatory assessment.
- Factory-specific severity weights and critical zones require production-owner approval.

2.4 Acceptance criteria

Criterion	Verification condition	Status
Classifier coverage	Each supported category resolves to a committed subtype classifier artifact	Met
Defect-only contract	Good is excluded from subtype label sets	Met
Evaluation	Accuracy, macro F1, class report, and confusion matrix are recorded	Met
Portability	Promoted CNN classifiers execute from ONNX; cable has compact forest runtime	Met
Uncertainty	Missing/conflicting classifier evidence returns unknown_defect with a reason	Met
Severity coverage	All classifier labels have explicit risk mappings	Met
QA policy	Pass, Review, and Fail decisions follow configured thresholds	Met
Software tests	Focused classifier, severity, prediction, and preprocessing tests pass	Met

3. Position in the Inspection Pipeline

Milestone 3 consumes the binary and spatial evidence produced by Milestone 2. It does not repeat anomaly detection. The classifier receives the selected category, image, and defect mask. Severity receives the defect type, defect geometry, critical-zone status, and detection confidence. The quality policy converts the score into a release action.

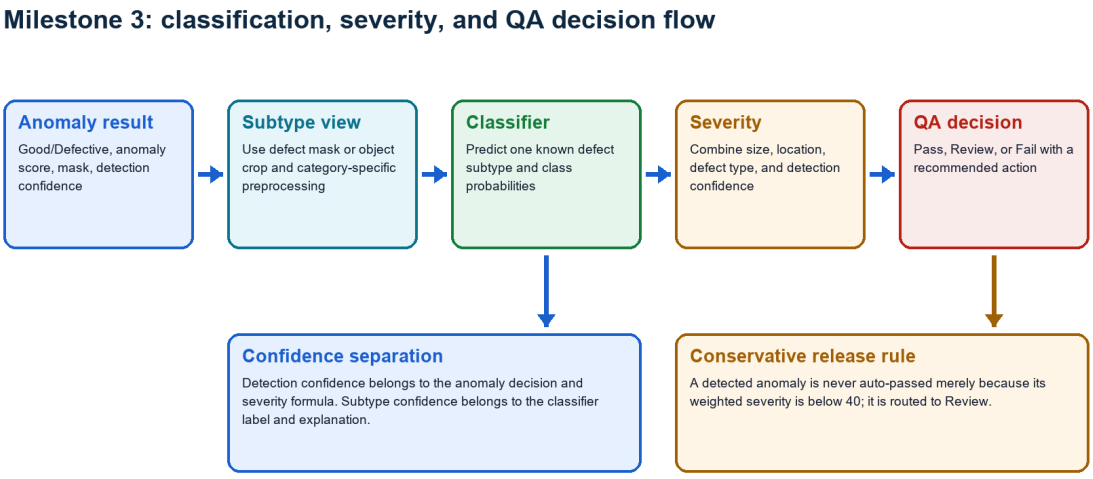

Figure 1. Milestone 3 classification, severity, and QA decision flow
Source: Generated from the active inference, classifier, and severity control flow.

3.1 Runtime contract between milestones

Boundary	Fields and responsibility
Input from Milestone 2	category, is_defective, anomaly score, detection confidence, anomaly map, predicted mask
Classifier output	defect_type, classification_confidence, class_probabilities, classifier_engine or error
Geometry output	defect area ratio, centroid, detected regions, critical-zone overlap
Severity output	weighted component scores, total score, level, recommendation
QA output	Pass, Review, or Fail plus a release/rework recommendation
Handover to Milestone 4	backend-ready dictionary, explainability notes, and display fields

4. Classification Dataset and Problem Definition

4.1 Why classification is defect-only

Good/Defective is already decided by the anomaly detector. Including good as a subtype would duplicate that decision and create contradictory cases. Therefore every subtype classifier is trained only from labelled defect folders, and every stored bundle records `defect_only=true`. If the anomaly detector says Good, the runtime returns defect_type=good without running a defect classifier.

4.2 Dataset construction

1. Select the MVTec product category supplied by the inspection request.
2. Load labelled images from category/test/<defect_subtype> and exclude test/good.
3. Resolve the matching ground-truth mask when available.
4. Build a classifier view from the full object, defect crop, or ROI evidence required by the feature mode.

5. Preserve the category label order in the model artifact and metadata.

4.3 Coverage

Category	Subtype count	Defect-only labels
bottle	3	broken_large, broken_small, contamination
cable	8	bent_wire, cable_swap, combined, cut_inner_insulation, cut_outer_insulation, missing_cable, missing_wire, poke_insulation
capsule	5	crack, faulty_imprint, poke, scratch, squeeze
carpet	5	color, cut, hole, metal_contamination, thread
grid	5	bent, broken, glue, metal_contamination, thread
hazelnut	4	crack, cut, hole, print
leather	5	color, cut, fold, glue, poke
metal_nut	4	bent, color, flip, scratch
pill	7	color, combined, contamination, crack, faulty_imprint, pill_type, scratch
screw	5	manipulated_front, scratch_head, scratch_neck, thread_side, thread_top
tile	5	crack, glue_strip, gray_stroke, oil, rough
toothbrush	1	defective
transistor	4	bent_lead, cut_lead, damaged_case, misplaced
wood	5	color, combined, hole, liquid, scratch
zipper	7	broken_teeth, combined, fabric_border, fabric_interior, rough, split_teeth, squeezed_teeth

Evaluation boundary: Most classical classifier metrics are out-of-fold predictions from the labelled MVTec test defect folders. Capsule and wood use stratified holdout evaluation. These are project validation records, not the official MVTec anomaly-detection benchmark and not independent factory acceptance results.

5. Feature Engineering and Runtime Architecture

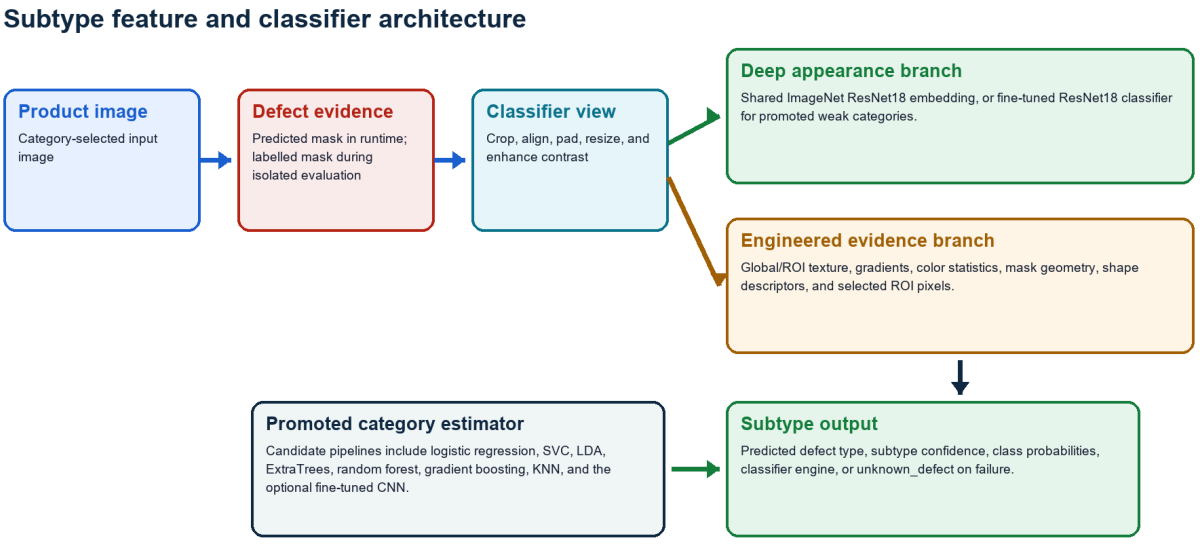

Figure 3. Subtype feature and classifier architecture
Source: Generated from `ml/classifier.py`, `ml/cnn_classifier.py`, and `ml/defect_classifier.py`.

5.1 Feature modes

Feature mode	Evidence represented	Current use
global_texture	Shared ResNet18 embedding plus global texture statistics	Bottle and toothbrush
global_roi_texture	Global embedding plus mask-guided ROI texture and geometry	Most structured categories
global_roi_pixel_texture	Embedding, ROI texture/shape, and selected ROI pixel evidence	Grid
handcrafted_roi_shape_texture	Portable OpenCV texture, gradient, color, ROI, and shape features	Cable
cnn_heatmap_object_crop	Fine-tuned ResNet18 on full/object/defect views	Capsule and wood

5.2 Runtime priority

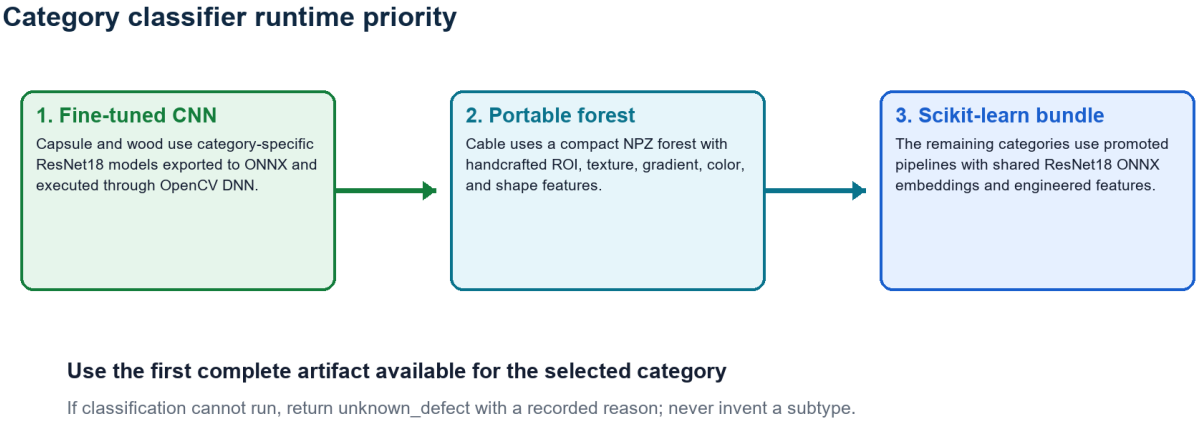

Figure 2. Category classifier runtime priority
Source: Actual priority implemented by `classify_defect_type`.

5.3 Portable artifacts

Artifact	Path	Size/count	Role
Shared ResNet18 feature model	models/inference/resnet18_features.onnx	44.69 MB	Deep embedding for scikit-learn bundles
Capsule CNN	models/categories/capsule/cnn_defect_classifier.onnx	44.71 MB	Fine-tuned ONNX subtype classifier
Wood CNN	models/categories/wood/cnn_defect_classifier.onnx	44.71 MB	Fine-tuned ONNX subtype classifier
Cable compact classifier	models/categories/cable/defect_classifier_runtime.npz	173 KB	Portable forest and scaling data
Category bundles	models/defect_classifier.pkl and models/categories/*/*.pkl	15 artifacts	Promoted production estimators and metadata

6. Training, Evaluation, and Model Promotion

6.1 Classical classifier path

6. Extract the configured global, ROI, texture, shape, or pixel features.
7. Choose a stratified fold count supported by the smallest subtype class.
8. Compare candidate estimators using mean macro F1, with accuracy as secondary evidence.
9. Generate out-of-fold predictions for the selected estimator and calculate the confusion matrix and class report.
10. Refit the promoted estimator on all labelled defect images and store the bundle and metadata.

Candidate family	Models considered
Linear	Logistic regression with several C values; shrinkage LDA; SGD log-loss
Kernel/distance	RBF SVC and K-nearest neighbors
Ensemble	ExtraTrees, random forest, and gradient boosting
Single-subtype	Constant DummyClassifier when the category has only one known defect subtype

6.2 Fine-tuned CNN path

- Use a 70/30 stratified defect-only split.
- Load ImageNet ResNet18 weights; freeze early layers and train layer4 plus a dropout classification head.
- Apply small brightness/contrast jitter and affine augmentation.
- Use weighted sampling and weighted cross-entropy with 0.05 label smoothing.

- Select the best epoch by holdout macro F1 with early stopping.
- Export the promoted model to ONNX opset 17 and execute through OpenCV DNN.

6.3 Promotion controls

Control	Behavior
Category-specific comparison	A candidate replaces the current classifier only when its recorded macro F1 improves, unless explicitly forced
Artifact completeness	ONNX requires both model and JSON metadata; compact forest requires a valid NPZ
Registry update	Only the promoted artifact path is added to category_model_registry.json
Runtime parity	Tests compare compact forest probabilities with the original scaled scikit-learn pipeline
Defect-only invariant	Model artifacts and tests verify that good is absent from subtype labels

7. Category Coverage and Classifier Metrics

7.1 Aggregate view

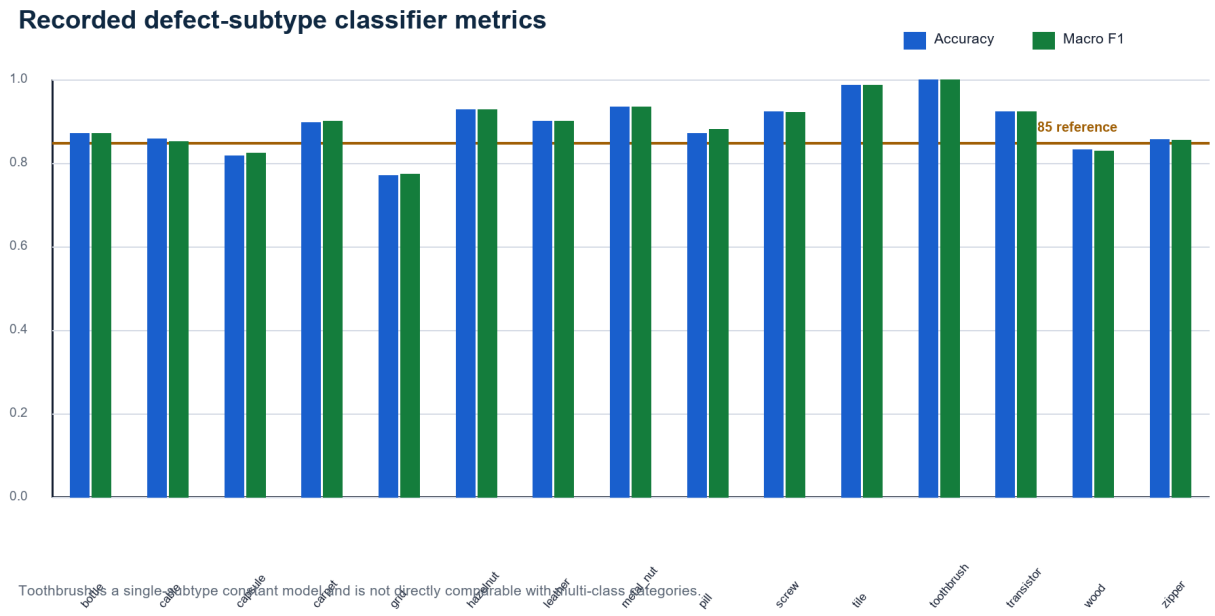

Figure 4. Recorded defect-subtype classifier metrics by category
Source: Current category model metadata; blue is accuracy and green is macro F1.

Category	Active engine	Classes	Accuracy	Macro F1	Selected estimator
bottle	Scikit-learn bundle	3	0.8730	0.8719	extra_trees
cable	Portable forest	8	0.8587	0.8529	random_forest
capsule	ResNet18 ONNX	5	0.8182	0.8250	resnet18_finetuned_weighted_cross_entropy
carpet	Scikit-learn bundle	5	0.8989	0.9009	lda_shrinkage
grid	Scikit-learn bundle	5	0.7719	0.7742	random_forest
hazelnut	Scikit-learn bundle	4	0.9286	0.9295	extra_trees
leather	Scikit-learn bundle	5	0.9022	0.9017	extra_trees
metal_nut	Scikit-learn bundle	4	0.9355	0.9357	lda_shrinkage
pill	Scikit-learn bundle	7	0.8723	0.8828	lda_shrinkage
screw	Scikit-learn bundle	5	0.9244	0.9233	extra_trees
tile	Scikit-learn bundle	5	0.9881	0.9875	logistic_c0.01
toothbrush	Scikit-learn bundle	1	1.0000	1.0000	constant
transistor	Scikit-learn bundle	4	0.9250	0.9246	logistic_c0.01
wood	ResNet18 ONNX	5	0.8333	0.8294	resnet18_finetuned_weighted_cross_entropy
zipper	Scikit-learn bundle	7	0.8571	0.8568	svc_rbf_c5

7.2 Interpretation

- All-category mean accuracy is 0.8925; mean macro F1 is 0.8931.
- Excluding the single-subtype toothbrush constant model, the 14 multi-class categories average 0.8848 accuracy and 0.8854 macro F1.
- Eleven of fourteen multi-class categories record macro F1 at or above 0.85.
- Grid is the weakest recorded classifier at macro F1 0.7742; capsule records 0.8250 and wood 0.8294.
- Tile has the strongest comparable multi-class result at macro F1 0.9875.
- Toothbrush accuracy/F1 of 1.0 reflects one known defect subtype and must not be interpreted as a difficult multi-class result.

7.3 Cable class-level evidence

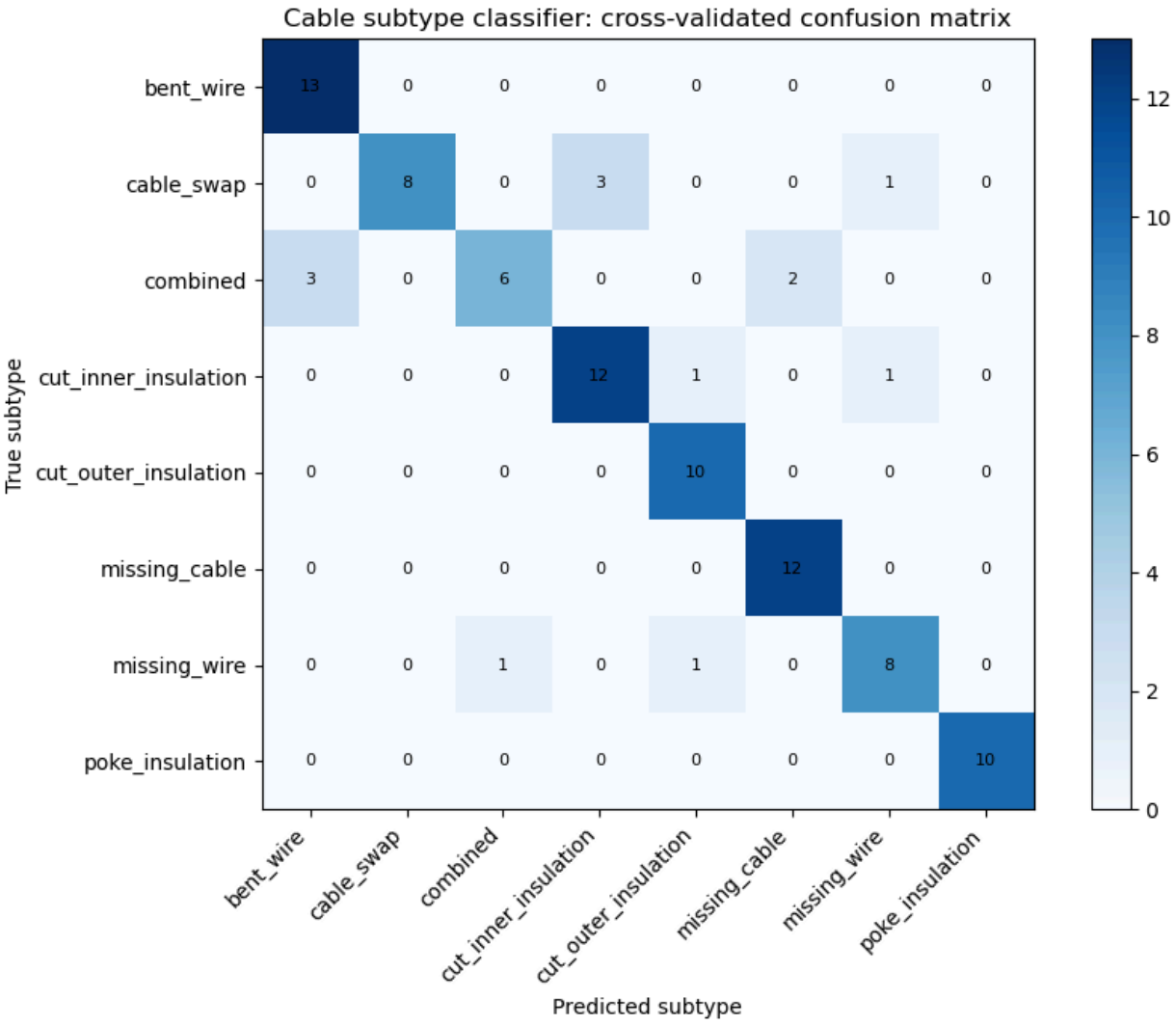

Figure 5. Cable subtype classifier cross-validated confusion matrix
Source: Executed Notebook 04 using recorded out-of-fold predictions.

Cable records accuracy 0.8587 and macro F1 0.8529 across eight subtypes. Bent wire, cut outer insulation, missing cable, and poke insulation are recovered cleanly in the recorded matrix. Most errors are concentrated among visually related combined, cable-swap, inner-insulation, missing-wire, and missing-cable patterns.

8. Practical Defect Subtype Demonstration

8.1 Demonstration design

Notebook 04 selects one real labelled MVTec defect image from cable, capsule, bottle, and wood. It supplies the labelled mask to isolate and demonstrate the subtype classifier itself. In the complete application runtime, the predicted anomaly mask is supplied instead. The isolated demonstration therefore proves classifier execution and label handling, not end-to-end localization quality.

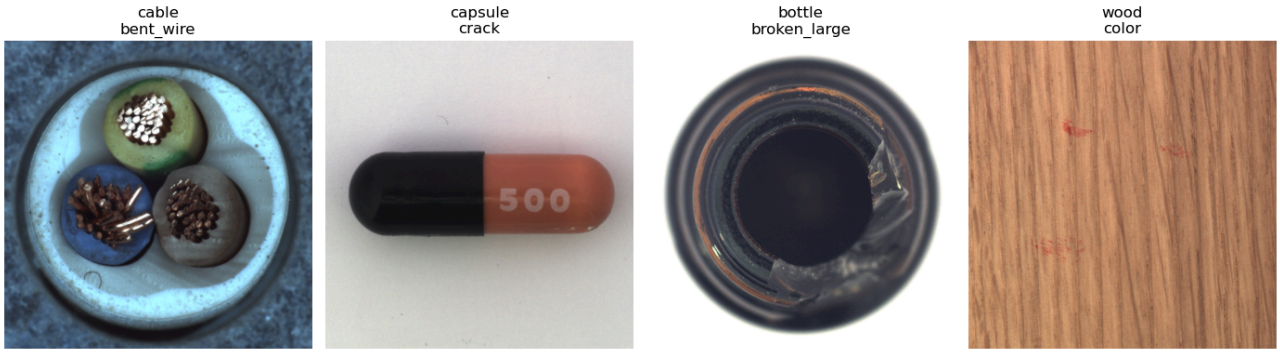

Figure 6. Practical subtype predictions on real MVTec images
Source: Executed Notebook 04; displayed labels are the expected defect folders.

Category	Expected	Predicted	Subtype confidence	Engine	Interpretation
cable	bent_wire	bent_wire	0.4133	portable_forest	Correct, but low subtype confidence
capsule	crack	crack	0.9922	fine_tuned_resnet18_onnx	Correct
bottle	broken_large	broken_large	0.9967	scikit_learn_bundle	Correct
wood	color	color	0.9944	fine_tuned_resnet18_onnx	Correct

What the demonstration proves: All three runtime paths execute from committed artifacts and return the expected subtype on the selected images. The cable result also demonstrates why subtype confidence must remain visible: the label is correct, but 41.33% confidence is not strong enough to present as certainty.

9. Confidence and Uncertainty Handling

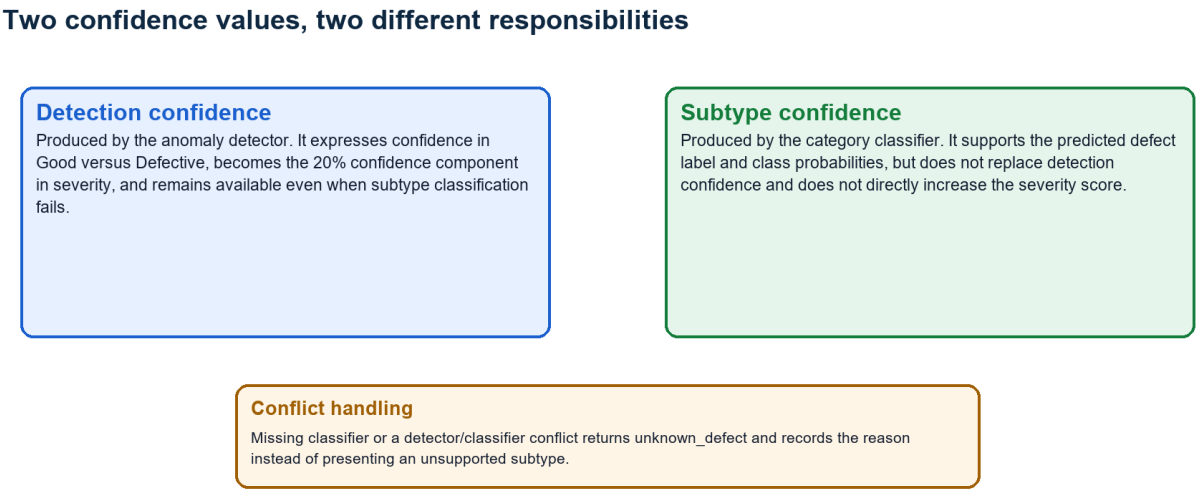

Figure 7. Detection confidence and subtype confidence separation
Source: Implemented by `classify_prediction`, `inspect_image`, and explainability output.

9.1 Why separation matters

An anomaly detector can be confident that an image is defective while the subtype classifier is uncertain about which defect is present. Combining those values would overstate subtype certainty or weaken the binary defect decision. VisionInspect AI therefore uses detection confidence in severity and reports subtype confidence only with the predicted label and class-probability distribution.

9.2 Unknown-defect behavior

Condition	Returned type	Safety behavior
Classifier artifact missing	unknown_defect	Record artifact path error; preserve binary defect decision
Classifier runtime error	unknown_defect	Record exception text; require manual classification
Detector says Defective but classifier says good	unknown_defect	Record conflicting decisions; do not reuse the classifier's good probability
Classifier succeeds	Known subtype	Store subtype confidence and complete class probabilities
Detector says Good	good	Do not invoke a defect subtype classifier

> **Current limitation:** The runtime does not yet apply a category-specific minimum subtype-confidence threshold. Low-confidence labels remain visible through their probability, while the conservative Review/Fail decision continues to depend on anomaly evidence and severity.

10. Severity Scoring Framework

10.1 Formula

> Severity Score = (Size x 0.30) + (Location x 0.25) + (Defect Type x 0.25) + (Detection Confidence x 0.20)

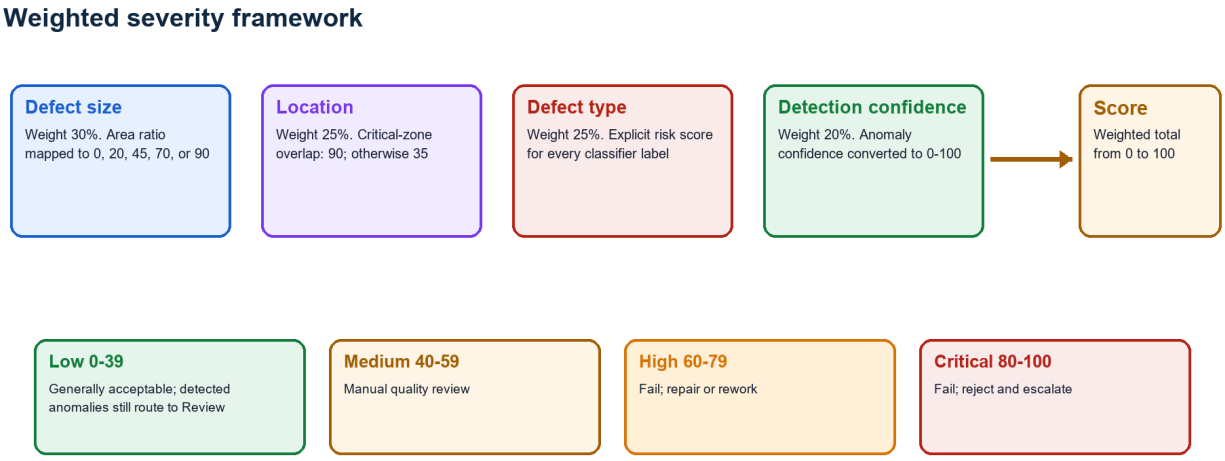

Figure 8. Weighted severity framework
Source: Implemented by `ml/severity.py`.

10.2 Component rules

Component	Weight	Current scoring rule
Size	30%	Area <=0: 0; <0.5%: 20; <2%: 45; <5%: 70; otherwise 90
Location	25%	Critical-zone overlap: 90; non-critical location: 35
Defect type	25%	Explicit policy value from 0 to 95 for all active classifier labels
Detection confidence	20%	Detector confidence converted to a bounded 0-100 score

10.3 Severity levels

Level	Score	Default recommendation
Low	0-39	Product generally acceptable; detected anomaly still routes to Review
Medium	40-59	Inspection review required
High	60-79	Repair or rework recommended
Critical	80-100	Reject product and trigger quality inspection workflow

10.4 Worked examples

Case	Severity score	Level	Direct level decision	Final runtime decision
Normal product	0.00	Low	Pass	Pass
Small scratch	37.90	Low	Pass	Review because a defect was detected
Contamination	53.20	Medium	Review	Review
Large critical crack	92.65	Critical	Fail	Fail

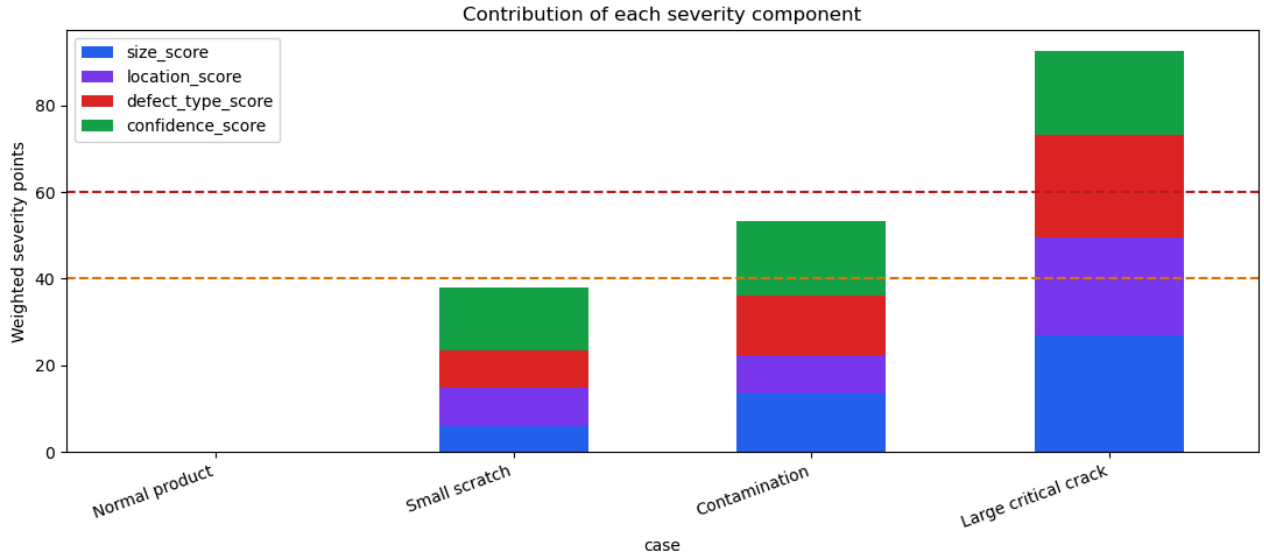

Figure 9. Severity component contribution examples
Source: Executed Notebook 06; dashed lines show Review and Fail thresholds.

11. Quality Decision Policy

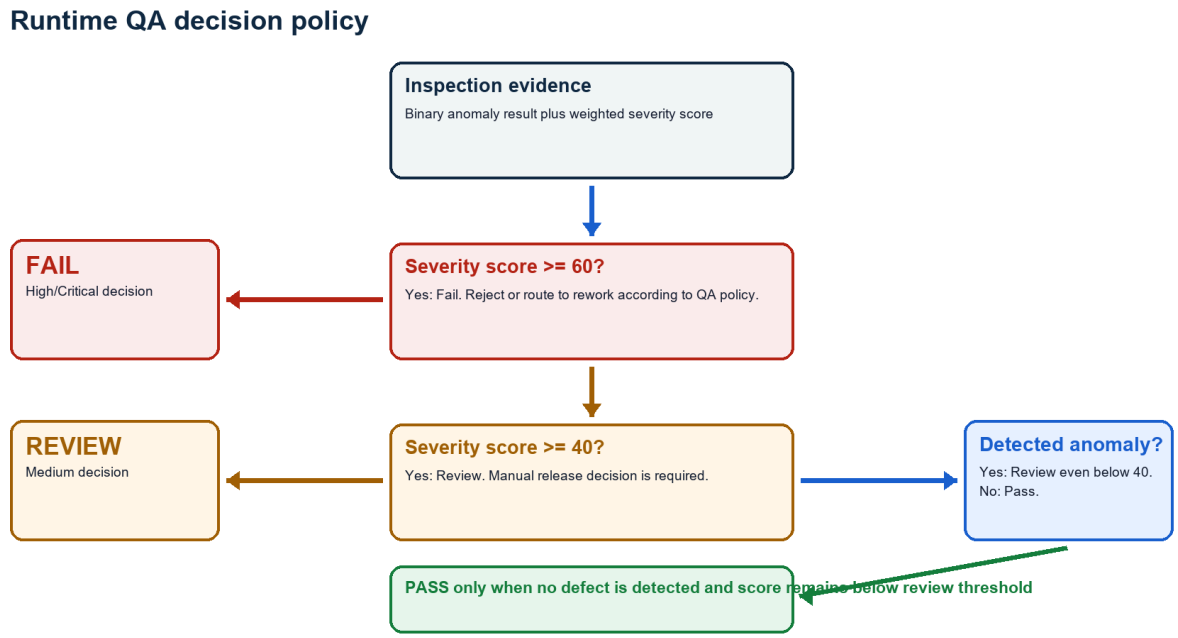

Figure 10. Runtime QA decision policy
Source: Implemented by `apply_quality_decision` with configurable review and fail thresholds.

11.1 Decision order

11. If severity score is at least the fail threshold (default 60), return Fail.
12. Otherwise, if severity score is at least the review threshold (default 40), return Review.
13. Otherwise, if an anomaly was detected, return Review even though the weighted score is Low.
14. Return Pass only when no defect is detected and the score remains below the review threshold.

11.2 Recommended actions

Decision	Runtime recommendation	Operational meaning
Pass	Product generally acceptable	Release subject to plant policy
Review	Manual quality review required before release	Engineer inspects image, heatmap, metadata, and confidence
Fail	Reject product or send to rework based on QA policy	Create rejection/rework workflow in the application

12. Practical Severity and QA Demonstration

12.1 Executed example

Notebook 06 executes the portable bottle pipeline on `bottle/test/broken_large/000.png` and then exposes only the fields needed to explain classification, severity, and the quality decision. The example uses a configured center critical zone.

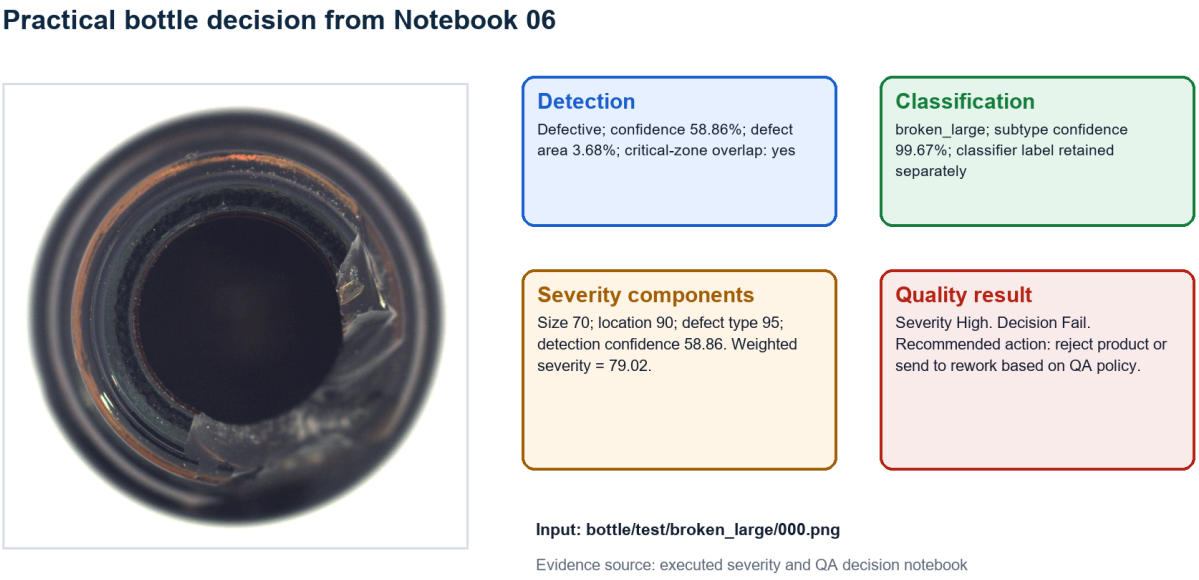

Figure 11. Practical bottle severity and QA decision
Source: Executed Notebook 06 using a real MVTec broken-large bottle image.

Field	Observed value	Interpretation
Binary prediction	Defective	Anomaly score exceeded the active threshold
Subtype	broken_large	Subtype confidence 99.67%
Detection confidence	58.86%	Used as the severity confidence component
Defect area	3.68%	Size score 70
Critical location	True	Location score 90
Defect-type risk	95	broken_large policy mapping
Severity	79.02 - High	Below Critical, above Fail threshold
Decision	Fail	Reject or route to rework according to QA policy

> **Decision trace:** The 99.67% subtype confidence does not become the confidence severity input. The score correctly uses the 58.86% detection confidence, preserving the distinction between confidence that a defect exists and confidence about its name.

13. Testing and Reproducibility

13.1 Focused automated tests

The focused suite validates stored classifier artifacts, portable ONNX priority, compact forest probability parity, feature reuse, confidence separation, unknown-defect handling, severity mappings, quality decisions, and classifier-view preprocessing. Twenty tests pass.

```
PS C:\VisionInspectAI-team> python -m pytest
tests/test_classifier_artifact.py tests/test_severity.py
tests/test_prediction.py tests/test_preprocessing.py -q
--basetemp tmp/pytest_m3_run_20260815 -p no:cacheprovider

....................                                    [100%]
20 passed in 4.86s

Validated: classifier artifacts, ONNX priority, compact forest parity,
confidence separation, severity mapping, QA policy, and preprocessing.
```

Figure 12. Focused Milestone 3 automated test result
Source: Focused automated test command output.

Test area	Behavior protected
Classifier artifact	Bottle and category bundles load with labels and defect-only metadata
ONNX priority	Promoted CNN artifact is selected before lower-priority classifiers
CNN inference	Capsule and wood ONNX artifacts return valid probabilities
Compact forest parity	Portable NPZ probabilities match the scaled scikit-learn pipeline
Feature reuse	Precomputed global features avoid duplicate deep extraction
Confidence contract	Detection and subtype confidence remain separate
Unknown defect	Missing classifier does not invent a defect subtype
Severity mapping	Every stored classifier label has an explicit risk score
Good result	Good prediction has zero defect severity and Pass
Preprocessing	Mask-guided classifier crop remains correctly shaped

13.2 Notebook reproduction

```
python -m pip install -r requirements-ai.txt
$env:MVTEC_DATASET_ROOT = 'C:\path\to\mvtec_anomaly_detection'
jupyter nbconvert --to notebook --execute --inplace notebooks/04_defect_subtype_classification.ipynb
jupyter nbconvert --to notebook --execute --inplace notebooks/06_severity_and_qa_decision.ipynb
```

Both notebooks show tables, plots, class probabilities, and decision evidence inline. They do not export external screenshots, CSV reports, or chart files.

14. Risks, Limitations, and Mitigations

Risk	Impact	Mitigation
Validation provenance	Most classical metrics use out-of-fold predictions on labelled MVTec test defects	Treat as project validation and collect an untouched factory acceptance set
Mask dependence	Subtype features can degrade when the predicted anomaly mask is inaccurate	Evaluate classifier performance with predicted masks, not only labelled masks
Low subtype confidence	Best class can be weak even when the label is correct	Add category-specific abstention thresholds and route uncertain cases to manual classification
Unseen subtype	Closed classifiers cannot name a defect outside their label set	Use unknown_defect, retain heatmap evidence, and add labelled examples before retraining
Single-subtype metric	Toothbrush 1.0 score is structurally easy and not comparable	Report the constant-model protocol explicitly
Weak categories	Grid, capsule, and wood remain below 0.85 macro F1	Collect more examples, improve masks/crops, and compare calibrated classifiers
Probability calibration	Classifier probabilities may be overconfident	Measure calibration error and apply Platt/isotonic/temperature scaling
Severity policy	Generic weights may not match every product's safety impact	Make risk maps and critical zones product-owner controlled and versioned
Artifact size	Three ResNet18 ONNX files consume about 134 MB	Share the common backbone where practical and evaluate quantization after parity testing

14.1 Controls already implemented

- Category selection prevents cross-product classifier use.
- Good is excluded from defect subtype labels.
- Unknown-defect fallback records missing, failed, or conflicting classifier evidence.
- Detection and subtype confidence are returned as separate fields.
- Severity mappings cover every currently stored classifier label.
- Detected anomalies cannot be auto-passed solely because of a Low severity score.
- ONNX and compact artifacts are tested for runtime availability and parity.

15. Milestone Completion and Handover

15.1 Completed deliverables

Deliverable	Evidence	Status
Defect-only category classifiers	15 committed category artifacts	Complete
Subtype taxonomy	73 category-label pairs; 48 unique names	Complete
Portable CNN runtime	Capsule and wood ResNet18 ONNX artifacts	Complete
Portable forest runtime	Cable compact NPZ artifact	Complete
Classifier evaluation	Accuracy, macro F1, class reports, and confusion matrices	Complete
Confidence handling	Separate detection/subtype confidence and unknown-defect path	Complete
Severity framework	30/25/25/20 weighting and complete risk map	Complete
QA decision policy	Pass/Review/Fail rules and recommendations	Complete
Practical evidence	Four subtype examples and one complete severity decision	Complete
Focused testing	20 passed	Complete

15.2 Handover to Milestone 4

Milestone 3 hands forward a category-specific defect type, subtype confidence, class probabilities, weighted severity components, severity score and level, Pass/Review/Fail decision, recommended action, and uncertainty notes. Milestone 4 connects that contract to backend APIs, persistence, image/report storage, frontend review workflows, analytics, batch processing, and deployment-oriented validation.

15.3 Milestone outcome

> **Milestone conclusion:** VisionInspect AI now turns localized anomaly evidence into a named defect, an auditable severity calculation, and a conservative quality-control action. The implementation remains transparent about classifier confidence, evaluation provenance, weak categories, and the distinction between project policy and factory-certified acceptance rules.

References

[1] Scikit-learn documentation, model selection, pipelines, ensemble classifiers, classification metrics, and confusion matrices. https://scikit-learn.org/stable/

[2] PyTorch and Torchvision documentation, ResNet18, transfer learning, weighted sampling, and model export. https://pytorch.org/docs/stable/ and https://pytorch.org/vision/stable/

[3] OpenCV documentation, DNN ONNX inference and image processing. https://docs.opencv.org/

[4] ONNX documentation, portable neural-network representation. https://onnx.ai/

[5] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, 'MVTec AD - A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection,' CVPR, 2019. https://www.mvtec.com/research-teaching/datasets/mvtec

[6] VisionInspect AI Member 4 source: ml/classifier.py, ml/cnn_classifier.py, ml/defect_classifier.py, ml/severity.py, ml/inference.py, model registry, and category metadata.

[7] VisionInspect AI executed notebooks 04_defect_subtype_classification.ipynb and 06_severity_and_qa_decision.ipynb.

Appendix A. Defect Subtype Taxonomy

Category	Count	Supported subtype labels
bottle	3	broken_large, broken_small, contamination
cable	8	bent_wire, cable_swap, combined, cut_inner_insulation, cut_outer_insulation, missing_cable, missing_wire, poke_insulation
capsule	5	crack, faulty_imprint, poke, scratch, squeeze
carpet	5	color, cut, hole, metal_contamination, thread
grid	5	bent, broken, glue, metal_contamination, thread
hazelnut	4	crack, cut, hole, print
leather	5	color, cut, fold, glue, poke
metal_nut	4	bent, color, flip, scratch
pill	7	color, combined, contamination, crack, faulty_imprint, pill_type, scratch
screw	5	manipulated_front, scratch_head, scratch_neck, thread_side, thread_top
tile	5	crack, glue_strip, gray_stroke, oil, rough
toothbrush	1	defective
transistor	4	bent_lead, cut_lead, damaged_case, misplaced
wood	5	color, combined, hole, liquid, scratch
zipper	7	broken_teeth, combined, fabric_border, fabric_interior, rough, split_teeth, squeezed_teeth

The registry contains 73 category-specific labels representing 48 unique names. Shared names such as crack, color, cut, hole, scratch, and combined are scored consistently by the severity policy.

Appendix B. Code and Artifact Inventory

Type	Path	Milestone 3 responsibility
Source	ml/classifier.py	Feature extraction, candidate estimators, selection, training, and portable forest export
Source	ml/cnn_classifier.py	Fine-tuned ResNet18 training, augmentation, evaluation, and ONNX export
Source	ml/defect_classifier.py	Runtime priority, ONNX/forest/bundle inference, probabilities, and caching
Source	ml/severity.py	Risk map, weighted formula, levels, and recommendations
Source	ml/inference.py	Confidence separation, unknown-defect handling, geometry, severity, and QA decision
Script	scripts/train_category_classifiers.py	Train and promote classical category subtype classifiers
Script	scripts/train_finetuned_cnn_classifiers.py	Train, compare, export, and promote optional CNN classifiers
Artifact	models/category_model_registry.json	Classifier path and runtime selection per category
Artifact	models/inference/resnet18_features.onnx	Portable shared deep feature extractor
Artifact	models/categories/capsule/cnn_defect_classifier.onnx	Capsule fine-tuned subtype model
Artifact	models/categories/wood/cnn_defect_classifier.onnx	Wood fine-tuned subtype model
Artifact	models/categories/cable/defect_classifier_runtime.npz	Cable compact forest runtime
Artifact	models/defect_classifier.pkl and category bundles	Promoted estimator, feature mode, labels, and metadata
Notebook	notebooks/04_defect_subtype_classification.ipynb	Coverage, metrics, confusion matrix, and live subtype predictions
Notebook	notebooks/06_severity_and_qa_decision.ipynb	Formula, examples, policy, and practical decision
Test	tests/test_classifier_artifact.py	Classifier runtime and artifact behavior
Test	tests/test_severity.py	Formula, levels, Good behavior, and label-map coverage
Test	tests/test_prediction.py	Confidence contract and unknown-defect behavior
Test	tests/test_preprocessing.py	Mask-guided classifier-view preprocessing

Appendix C. Evidence Index

Evidence	Source	What it proves
Pipeline diagram	Inference/classifier/severity source	Stage boundaries and data flow
Runtime priority diagram	ml/defect_classifier.py	ONNX, compact forest, and bundle selection
Feature architecture	Classifier sources	Deep and engineered evidence used by subtypes
Metric chart/table	Category metadata	All-category accuracy and macro F1
Cable confusion matrix	Notebook 04	Class-level errors across eight cable subtypes
Four practical subtype images	Notebook 04	Three runtime engines returning expected labels
Confidence diagram	ml/inference.py	Detection confidence remains separate from subtype confidence
Severity formula/chart	Notebook 06 and severity source	Weighted score and worked examples
QA decision flow	ml/inference.py	Pass, Review, Fail, and conservative anomaly override
Practical bottle decision	Notebook 06	Traceable High severity and Fail decision
Focused test output	Pytest	20 passing classifier/severity/prediction/preprocessing tests

End of Milestone 3 documentation.